Generating a Kubernetes Deployment Set from a Project-Intent Domain Model

Project Task 2: Model Transformations
Project Type: Code Generation
October 16, 2026

Joris Jonkers (S1829157)

Model Driven Engineering
University of Twente

Contents

1 Purpose of the Project **1**

1.1 Goals . . . 1

1.2 Role of the Transformations . . . 1

1.3 Refinement of Tasks 0 and 1 . . . 2

1.4 Assumptions and Scope . . . 2

2 Alternatives and Justification **3**

2.1 Existing Kubernetes Modelling Tools . . . 3

2.2 The Transformation Language . . . 3

2.3 The Structure of the Transformation . . . 4

2.3.1 Lowering Before the Resolution . . . 4

2.3.2 One Project per Run, All Projects as Input . . . 4

2.3.3 Typed Extents . . . 4

2.3.4 Stops, Not Defaults . . . 4

3 Solution Design **5**

3.1 The Lowering . . . 5

3.2 Interface, Mappings and Queries . . . 5

3.3 Worked Example . . . 7

3.4 The Derivation Rules . . . 8

3.4.1 The Process Mapping . . . 9

3.4.2 The Edge Rule . . . 9

3.4.3 Environment Variables and Placeholders . . . 10

3.4.4 Network Peers . . . 10

3.4.5 The Path Plan . . . 10

3.5 The Trace and End Sections . . . 10

3.6 The Black-Box Library . . . 10

3.7 The Estate Pass . . . 11

3.8 Stops and Running the Transformations . . . 11

4 Tests and Evaluation **11**

4.1 Results . . . 12

4.1.1 Revision Evidence . . . 12

4.1.2 Generated-File Evidence . . . 13

4.1.3 Thin Evidence . . . 13

4.1.4 Defects Found . . . 13

4.2 Limitations . . . 13

4.3 Reflection . . . 14

A Changes to the Task 1 Metamodels **15**

A.1 The Pinned Inputs Metamodel 15
A.2 Changes to the Target Metamodel 15
A.3 Changes to the Source Metamodel 21
A.4 Grammars for the Pinned Inputs 21

B The Hand-in Archive **22**
B.1 Running the Transformations 23

1 Purpose of the Project

1.1 Goals

The project turns short deployment documents into the Kubernetes files that deploy applications to an existing small cluster. Each `Application`'s repository holds one `Project Intent` document that states what the `Application` needs: its `Process`es, their memory and CPU limits, the ports they expose, the `Application`s they depend on, and the routes that reach them from outside. One `Platform Intent` document states what all `Application`s share: cluster entry points, health-check timings and backup policies. Task 1 defined the Ecore metamodels for both. The source metamodel covers the authored documents. The target metamodel covers the `Resolved Deployment`: every value the platform derives for an `Application`, and the file path each value writes to.

Task 2 builds the model-to-model transformation from source to `Resolved Deployment`. Its inputs are the `Project Intent` and `Platform Intent` models of every project in the cluster and the documents other processes publish; it outputs one `Resolved Deployment` per project for Task 3 to consume. The transformation makes the specification's derivation rules executable: every value in a `Resolved Deployment` is produced by a rule, and no author writes any value by hand.

Term	Meaning
`Process`	One program of an `Application`, running in its own pods.
`Dependency edge`	A declaration by one `Process` that it calls a named port of another `Application`.
`Pinned input set`	The read-only inputs besides authored documents: node contract, images lock, `ClusterState` snapshot, migration proofs, `Asset`s.
`Switchover`	How a new version replaces the old: blue/green (both run; traffic switches once the new is healthy), rolling, or stop-start.
`Release Gate`	The service that blocks a blue/green switch until every `Process` of the `Application` is healthy.
`Reconcile Unit`	A group of files that Flux applies together and in order.
`Revision`	A SHA-256 digest of an `Application`'s complete element, naming one release.
`Estate`	All `Project`s, the `Platform Intent` and the `Pinned input set` read by one run.
`Effective Intent`	`Project Intent` with every shared declaration lowered onto the `Process`es that hold it.

Table 1: Project Terms Used in This Report

The transformation has three goals. First, it must derive the same values as the TypeScript production implementation, which deploys the actual cluster. Both implementations run against the same oracle files and must produce identical results. Second, it must stop with an error message when it cannot derive a value, because a `Resolved Deployment` with a missing or fabricated value would look valid yet deploy the wrong configuration. Third, its output must be complete enough that the Task 3 templates only write values into their fixed positions. Any path, port or name chosen inside a template would hide a derivation from review.

1.2 Role of the Transformations

Figure 1 shows the pipeline. Xtext converts each document to a source model, and the Task 1 OCL constraints reject invalid ones. Three QVT-Operational transformations then run in sequence.

The first transformation, the lowering, copies declarations made once for an entire `Project` or `Application` down onto each `Process`. Its output is the `Effective Intent`, which still conforms to the source metamodel. Although it was implemented in Task 1, it was not reported there, so Section 3.1 covers it here. The second transformation, the resolution, reads the lowered models and the `Pinned input set`, then writes one `Resolved Deployment` per project. It is the main subject of this report. The third, the estate pass, runs once over all `Resolved Deployments` produced together and adds the files that do not belong to any single project (Section 3.7).

A small Java exporter writes the resolved dependency edges as JSON for comparison against the oracle. The Acceleo templates of Task 3 then read the `Resolved Deployments` and generate the Kubernetes files.

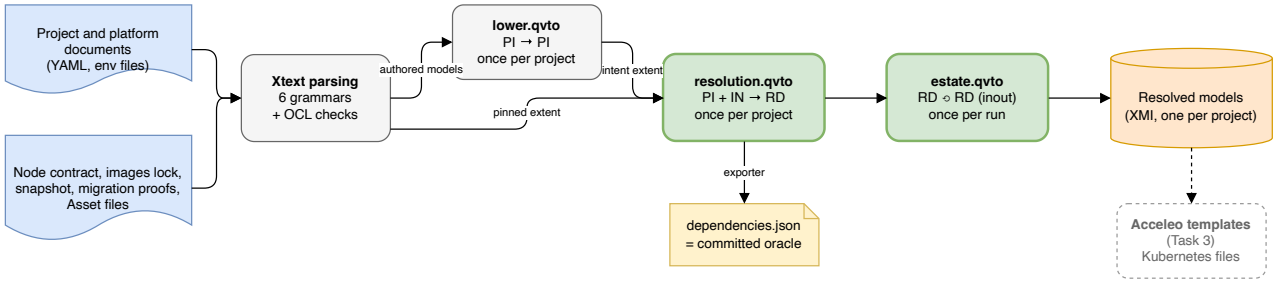


Figure 1: The Pipeline, With the Two Transformations Task 2 Adds in Bold Green

1.3 Refinement of Tasks 0 and 1

Task 0 treated the node contract, images lock and snapshot as plain inputs, while Task 1 only modelled the two authored documents. The resolution needs these as models, so a third metamodel now describes the `Pinned input set`, with its own Xtext grammar for each document (Appendix A.1). This metamodel lives in a separate package because no project author writes these documents.

Task 1 described three stages: authored, resolved and generated text. The lowering adds a fourth stage between authored and resolved, so each rule in the resolution reads values from a single level (Section 2.3.1).

Task 1 planned one resolved model for the entire cluster, because edges, routes and reverse dependencies require a cluster-wide view. Every run still reads every project. The output is now one `Resolved Deployment` per project, because the production implementation and the oracles are per project (Section 2.3.2); the per-`Application` projection from Task 1 is still present as the `ResolvedApplication` inside that model. The estate pass adds the one job that no single project's resolution can compute (Section 3.7).

Task 1's target metamodel only had a worked example for `minimal`. The other examples needed additional concepts: migrations, backup identities, API access, sidecar containers and twelve more file types. Appendix A lists every change.

1.4 Assumptions and Scope

The transformation assumes four conditions.

- **Valid input.** The Task 1 OCL constraints reject an invalid source model before the lowering runs, checking the entire document set of a run. A stop in the transformation is a safety net for facts the constraints do not yet cover (Section 3.8).
- **One cluster per run.** A run reads one `Platform Intent` and one node contract, both naming the same cluster. All concepts, examples and oracles derive from that single existing cluster.
- **The production implementation's choices.** Where the specification allows a choice, the transformation decides as the production TypeScript implementation does.
- **No live cluster access.** The transformation only reads the documents provided to it. Any fact known only to a running cluster, such as which node holds a particular volume, must arrive via the `ClusterState` snapshot captured by another process.

More Than One Cluster. The current implementation targets a single cluster, and every cluster-specific element is already a run input: the `Platform Intent`, the node contract and the `ClusterState` snapshot each name their cluster, and a `Project`'s environment files use a shared base plus a per-cluster overlay that the resolution selects via the `Platform Intent` cluster name (Section 3.4.3). Supporting a second cluster would therefore need a second run over that cluster's own three documents, and two additions beside it: a `Project Intent` would need to declare which clusters it targets, and the network policy of an edge to another cluster would need to allow an address range rather than a specific pod address. Such an edge would resolve through the `Platform Intent` external provider list, as the current edge from `auth` to an external mail server already does (Section 3.4.2).

Table 2 sets the scope of Task 2.

In scope	Out of scope
The resolution of eight examples: `minimal`, `auth`, `data`, `delivery`, `edge`, `knowledge-platform`, `observability` and `secrets`.	The `knowledge` example, which models a project being retired (Section 4.2).
Every derivation these examples use, from dependency edges and placement to backups, migrations, network peers and the path of every generated file.	The text of the generated files, which is Task 3.
The lowering's extensions, the estate pass, and the `Pinned input set` metamodel and grammars.	Applying the files to a cluster, and a second cluster.

Table 2: Scope of Task 2

2 Alternatives and Justification

2.1 Existing Kubernetes Modelling Tools

Three open-source tools were reviewed to see whether they could produce the `Resolved Deployment` from `Project Intent` and `Platform Intent`, or at least take part in the pipeline. Table 3 compares them.

Tool	What it does	Why it cannot replace the resolution	Where it could help
KubeDiagrams [1]	Draws diagrams from Kubernetes files or a live cluster, mapping kinds to icons.	It works from files to pictures, not from models to models, and derives nothing.	It could visualize the generated files to show links produced by a project document.
KubeOrch [2]	A drag-and-drop editor whose back end wires ports, fills templates, writes files and deploys.	It has no metamodel and no constraints; its rules are Go code, and it deploys directly, not via Flux.	Its graph notation could inform a project document editor.
kubert [3]	A command-line tool to switch cluster contexts and run commands on multiple clusters.	It is an operations tool and does not model or generate anything.	It could apply files to several clusters, which is out of scope.

Table 3: Existing Kubernetes Modelling Tools

None of the three derives values from authored declarations. KubeOrch comes closest: it resolves a link between two services to an address and port, similar to how the resolution resolves dependency edges. But it lacks a `Platform Intent`, node contract, images lock and snapshot, so it cannot derive placement, image digests, backups or network peers. Its output is not a model that OCL constraints or oracles can verify. Beside the transformation, the resolved dependency edges and network peers could feed a KubeDiagrams view of the Task 3 files for visual verification. A KubeOrch-style graphical editor could write `Project Intent` documents, since the editor and the transformation would work from the same Ecore metamodel.

2.2 The Transformation Language

Four languages were considered: ATL, QVT-Relations, QVT-Operational (QVTo) and Java with a rule library such as SiTra.

QVT-Relations was ruled out first. Its automatic traces and two-way synchronisation offer no benefit for a one-way derivation. In the cited experiments, one of its engines, medini QVT, ran into heap-space problems and became unresponsive on general-case input models with more than a hundred classes or a hundred attributes [5].

ATL, QVTo and Java were compared against the specific needs of this transformation. Most values in a `Resolved Deployment` do not derive from a single source element: a process's eligible machines depend on the node contract, a route's precedence on other routes, and a process's ingress peers on every process in the cluster that depends on it. ATL's matched rules fit one source element producing one target element, with the engine

choosing the order. The lookups this resolution needs are of another kind: resolving a dependency edge searches every project for the first process that provides the named surface, and the `Revision` has no source element of its own at all, since it digests the assembled application element. In ATL this work moves into called rules and helpers, which turns the transformation into imperative code without the pattern matching, and the values that need the whole model to exist first still have to be computed afterwards.

A Java transformation would be the easiest to debug and offers full control, but it requires maintaining the trace by hand and hides the transformation rules inside general-purpose code; some attendees of the cited discussion called model management in such code hard to maintain [4].

QVTo was chosen, in which the caller decides the order of execution explicitly and every reference from one created element to another goes through the trace. Its expressions use OCL, the same language Task 1's constraints already use. Among the cited survey's respondents it was the second most-used transformation language after ATL, and its users named its maturity and its support by a standard as reasons to use it [4]. The one operation QVTo cannot express directly, computing a SHA-256 hash, uses QVT's black-box mechanism: a Java library called through a contextual operation (Section 3.6).

Speed was not decisive. The cited experiment [5] measured QVTo more than five times slower than ATL, a result about the models of that experiment rather than a prediction for this cluster; what this cluster can bear is a question of capacity, which Section 4.2 returns to.

2.3 The Structure of the Transformation

Every derived value needs to be in a model that tests can compare, so derivation must happen in a model-to-model step before code generation, and the oracles can then verify values independently of the templates.

2.3.1 Lowering Before the Resolution

A single transformation from authored model to `Resolved Deployment` would work, but a lowering precedes the resolution. The lowering copies shared declarations down to each `Process`, producing `Effective Intent`, so the resolution reads values from one level instead of three and each rule is simpler. The chain also matches QVT specification Section 8.1.13, which composes a preparation transformation before the main one [6].

2.3.2 One Project per Run, All Projects as Input

One resolution over all projects at once could produce one `Resolved Deployment` for the whole estate. Instead, each run resolves one `Project` into its own model while reading every project's documents from the same input extent. Each model stays small and has one oracle to compare with, and the shared extent keeps the cluster-wide view that edges, routes and reverse dependencies need. One value, the digest over all Vault documents, needs the output of every project, so the estate pass, which runs over all resolved models together, adds it (Section 3.7).

2.3.3 Typed Extents

A rule also has to find the run's other documents. Using OCL's `allInstances()` scans everything loaded on each evaluation and gives results that depend on which files were loaded. Instead, the transformation selects each input once by type from two extents that hold exactly the run's documents, and every rule navigates from one of these properties (Section 3.2).

2.3.4 Stops, Not Defaults

A default for a value the resolution cannot derive would give a file that looks valid and deploys the wrong configuration, so the run stops instead, through a fatal assertion in a stop query (Section 3.8). The caller then has to handle the failure.

3 Solution Design

Task 2 uses three QVT-Operational transformations. The lowering (`lower.qvto`) is about 340 lines with 37 mapping operations and 8 queries. The main resolution (`resolution.qvto`) is about 1700 lines with 8 mapping operations and 140 queries. The estate pass (`estate.qvto`) is about 130 lines with no mappings but one helper.

Figure 2 shows the metamodels each transformation reads and writes. The three differ in kind, and all are one-way.

- The lowering is endogenous and horizontal: it reads and writes the source metamodel at the same abstraction level, producing a new model. One project goes in, one comes out.
- The resolution is exogenous and vertical, a refinement: it reads source and pinned-input metamodels and writes the target metamodel, taking several input models and producing one output.
- The estate pass is endogenous and in-place: it updates the resolved models of a run through an `inout` extent.

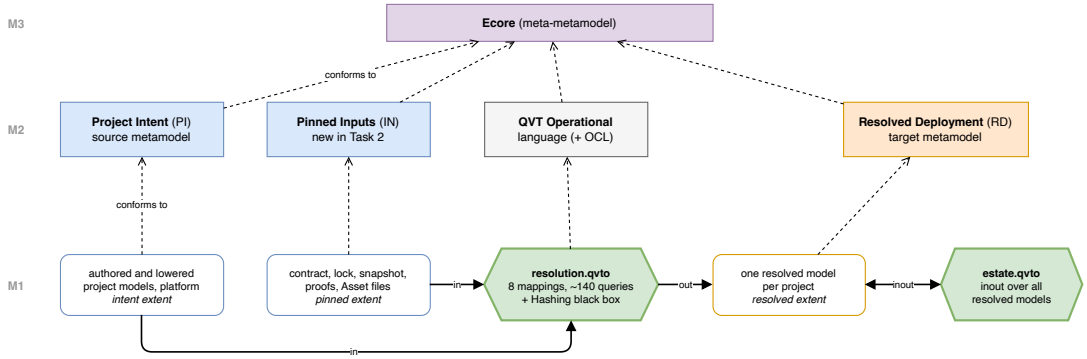


Figure 2: The Resolution and the Estate Pass in the Metamodel-Based Transformation Pattern

3.1 The Lowering

The lowering, `lower.qvto`, turns each authored `Project` into an `EffectiveProject`. Every `Process` carries all shared declarations that apply to it, while the `Project` and `Application` carry none. Task 2 extended the Task 1 implementation with API access and engine grants.

Its 37 mappings are nearly all simple copies; its queries do the merging. A list such as `secrets`, dependency edges, `Assets` or environment entries becomes the union of declarations from `Project`, `Application` and `Process` levels. The lowest level that declares an item wins. A single value such as startup budget or cutover comes from the lowest level that declares it. Memory and CPU remain only on the `Process`.

A mapping returns its cached result when called again on the same source, which would incorrectly share a single object among multiple `Processes`. To prevent this, every shared copy uses a mapping with the `Process` as a parameter (`copyFor(self)`), which keys the trace on the source-process pair. A migration is one of three subclasses, so its copy uses a disjuncts mapping that selects the subclass at runtime.

The lowering has no stops. The OCL constraints run before it, so every input it sees is valid.

3.2 Interface, Mappings and Queries

Example 1 shows the resolution's signature. It reads two extents from two metamodels and writes one extent of a third.

```
transformation resolution(in intent : PI, in pinned : IN, out resolved : RD)
        access library Hashing;
configuration property project : String;    -- the project this run resolves
property estate : Platform = intent.rootObjects()[Platform]->any(true);
property projects : Sequence(EffectiveProject) =
        intent.rootObjects()[EffectiveProject]->sortedBy(p | p.project)->asSequence();
property lock : ImagesLock = pinned.rootObjects()[ImagesLock]->any(true);
```

```
-- and likewise contract, snapshot, proofs and assetFiles
main() {
        projects->select(p | p.project = project)->any(true).map deployment();
}
```

Example 1: The Signature and Inputs of the Resolution, Abridged

The intent extent holds, for every project in the run, its authored model and its `EffectiveProject`, plus the `Platform Intent`. Authored models remain only because `Platform Intent` references them via Ecore cross-references. Every rule reads the `EffectiveProject` models.

The pinned extent holds the node contract, images lock, snapshot, each project's migration proof, and one `AssetFile` per configuration file. Two configuration properties provide non-derived inputs: the project name to resolve, and the schema package digest for provenance recording.

The seven properties select each input once by type from the root objects of its extent, and no rule calls `allInstances()`.

A query over every `Process` in the cluster uses `projects.applications.processes`, whose extent is exactly the run's documents. Projects are sorted by name so that the first provider of a service is consistent across runs and implementations.

QVTo provides two ways to create target objects. A mapping operation records its source and result in the trace. Other rules find that result later using `resolveone`. An object expression inside a query creates an object that no other rule needs to locate.

The transformation uses a mapping where another rule must reference the result. Each target type has exactly one mapping, so the type alone identifies the result: `p.resolveone(ResolvedProcess)` needs no `resolveIn`. No mapping uses a `when` guard or inherits from another; the caller selects what to map using `->select`.

Table 4 lists the eight mappings and their purpose.

Mapping	Creates, and why it is a mapping
`EffectiveProject::deployment`	The `ResolvedDeployment`. Its end section sets values that need the complete model.
`EffectiveProject::unit`	The project's `ReconcileUnit`, the group Flux applies together. Each `Application` refers to it.
`EffectiveProject::provenance`	The digest of every input and the hash over them. Its end section computes the hash.
`EffectiveApplication::application`	A `ResolvedApplication`. Generated files and the `Revision` refer to it.
`EffectiveApplication::gate`	The `ReleaseGate`'s inputs. The `Application` and its configuration refer to them.
`EffectiveApplication::migrationOf`	The database migration. Its network policy refers to it.
`Process::process`	A `ResolvedProcess`. Gate members, routes, scrapes, policies and generated files refer to it.
`Exposure::exposure`	A `ResolvedExposure`. The generated route file refers to it.

Table 4: Mapping Operations of the Resolution

Queries never change the model they read: each builds a value, and the caller decides where the result goes. Many attach to model types, such as `Grant::resolvedFor`.

A value query derives a single value, such as a deadline or an address. An object query builds a contained object, such as a probe or a backup plan. A spelling query turns a source enumeration literal into the string the target records, for example `AlertClass::businessHours` into `business-hours`. The exception is a stop query such as `unlocked`, which raises a fatal assertion that aborts an invalid run (Section 3.8).

The estate pass contains the sole QVTo helper, an operation with side effects that adds files to a model it updates in place.

Mappings run in a fixed order. The deployment mapping creates `ReconcileUnits` first, then `Applications`,

provenance and the path plan. Each `Application` maps its `Processes` before its `Exposures`, `ReleaseGate` and migration, because those refer to `Processes` through the trace.

3.3 Worked Example

The `minimal` example declares one `Application`, notes, with one `Process`, notes-api, in about 35 lines of YAML. Table 5 follows declarations through to derived values.

Declared	Rule	Derived in the resolved model
`image: notes-api`	`locked`	`ghcr.io/jorisjonkers-dev/notes/notes-api@sha256:9c1e...`, user/group 1000, from images lock
`startupBudget: 20s`	`deadlineOf`, startup probe	deadline 60s (three budgets); the startup probe polls the liveness target every 5s and fails after 4 attempts
`cutover: continuous`	`switchoverOf`, `gate`	blue-green; `Release Gate` with 1 member, deadline 60s, platform cadence, error/latency checks
`memory: 256Mi, cpu: 50m`	`eligible`	all 7 machines in the node contract, each tested for twice the request size for blue/green deployment
`audience: anonymous`	`tierFor`, `exposure`	entry point public-frankfurt with TLS or ACME certificates, proxy traefik-public in the edge-system project
route /, prefix	`precedenceOf`, `middlewareOf`	precedence 1; security headers with strict content policy
`runtime: node`	`profileOf`	environment variables `OTEL_SERVICE_NAME`, `PORT=8080`, `DEPLOYMENT_ENVIRONMENT`, collector address from observability
scrape on http	`ingressOf`, `monitorOf`	ingress from traefik-public and prometheus on port 8080; pod monitor for blue/green
(nothing)	path plan	12 `Deliverable` entries, including `apps/notes/namespace.yaml` and `estate/edge/public-frankfurt/notes-public.yaml`

Table 5: Declarations of `minimal`, Followed Through the Resolution

Four values depend on other run documents: the image needs the lock, eligible machines need the node contract, the proxy needs the edge project, and the collector address needs the observability project.

Example 2 shows the authored `Process` in YAML, and Example 3 shows the `Process` the resolution assembled from it, in XMI.

```
- name: notes-api
  image: notes-api
  placement: { memory: 256Mi, cpu: 50m }
  startupBudget: 20s
  cutover: continuous
```

Example 2: The Authored `Process` notes-api, in YAML (Condensed)

```
<processes name="notes-api" deadline="60s" switchover="blue-green"
    image="ghcr.io/jorisjonkers-dev/notes/notes-api@sha256:9c1e5a4b..."
    uid="1000" gid="1000" memory="256Mi" cpu="50m">
  <startup period="5s" timeout="5s" failures="4"> ... </startup>
  <placement> <eligibleNodes>enschede-t1000-1</eligibleNodes> ... </placement>
</processes>
```

Example 3: The Resolved `Process` notes-api, in XMI (Abridged)

The declared `startupBudget: 20s` becomes the `deadline` field of the resolved `Process` through `deadlineOf`, 60s in Example 3, with the startup probe beside it. Through `switchoverOf`, the declared `cutover: continuous` sets the switchover field of the same `Process`, and the path plan then assigns that blue/green `Process`'s `Canary` its path, `apps/notes/notes/canary.yaml`, one of the twelve `Deliverable` entries of Table 5. The `gate` rule adds the `Release Gate` whose inputs the configuration map beside the `Canary` carries. Section 4.1 checks this whole model against the complete expected model of `minimal`, attribute by attribute.

3.4 The Derivation Rules

Table 6 lists the rules by target element. Each implements a derivation from the specification.

Target value	Rule	Derivation
Application		
reconcile unit and order	`unit`, `afterOf`	One unit per project; it follows the units of projects its edges reach, and the secrets unit if it holds a secret.
secret store	`synced`, `storeEndpoint`	The Secret Store address used when a secret, backup or migration needs it.
migration	`migrationOf`, `datastoreOf`	Runner image (from lock), tested schema (from proof), database address (from edge), owner credential and egress network.
release gate	`gate`, `switchesBlueGreen`	One member per blue/green `Process` with readiness; the slowest member deadline; platform cadence; checks where telemetry is exported.
Process		
image, user, group	`locked`	The lock entry for the image name.
switchover	`switchoverOf`	stop-start for interrupted `Process`, rolling for platform delivery tools, blue-green otherwise.
deadline	`deadlineOf`	Three startup budgets rounded down; budget rounded up for prepare `Process`; 600s without budget.
probes	`Probe::cadenced`	Declared target with platform period, timeout and failures. The startup probe polls the liveness target every 5s until the budget runs out.
eligible machines	`eligible`	Every machine that fits the memory and sidecars twice for continuous deployment, with required architecture, site, capability, GPU and disk.
environment	`environmentOf`, `entryFor`	Base environment overlaid by cluster overlay, with placeholders replaced and runtime profile variables; result is sorted.
secrets	`Grant::resolvedFor`	Key-value grant keeps the path and keys and gets a Kubernetes Secret. A transit or database grant becomes the Vault paths its policy covers.
volumes and backups	`backupFor`, `backupEgress`	For recoverable and irreplaceable volumes: platform schedule and retention, backup image for engine, backup identity and claim, connection, credential, off-cluster copy.
configuration files	`Asset::resolvedFor`	The file text, named after the process, the file and a digest prefix, so a changed file gets a new name.
dependency edges	`edgeOf`	The first `Process` in the cluster with the `Application` and port the edge names, with address and network peer; otherwise external providers.
Exposure		
entry point	`tierFor`	First entry point carrying the audience.
route precedence	`rankOf`, `precedenceOf`	Exact routes come before prefix routes, and longer prefixes before shorter ones.
middleware	`middlewareOf`	Authentication for authenticated routes, security headers, and redirect if declared.

Target value	Rule	Derivation
Network policy, per Process		
ingress	`ingressOf`	Proxy of each route entry point, metrics collector if the `Process` is scraped, every `Process` with edge to its ports, its backup identity, migrations of databases it holds, delivery events receiver.
egress	`egressOf`	Cluster DNS, and the Secret Store if the `Process` holds a secret.
Deployment		
provenance	`provenance`	One digest per input, and a hash over all of them.
revision	end of deployment	Digest of each `Application`'s complete element (Section 3.6).
generated files	`projectFiles`, `applicationFiles`, `edgeFiles`, `vaultFiles`	Path of every file, with elements it is generated from (Section 3.4.5).

Table 6: Derivation Rules, by Target Element

3.4.1 The Process Mapping

Example 4 shows the start of the `Process` mapping. The init section looks up the lock entry once, so assignments reuse it. Each assignment calls a query from Table 6. An optional feature is assigned only when its condition holds. Assigning an unset value would mark the feature as set, and the model would then state a value that no rule derived.

```
mapping PI::Process::process(in app : EffectiveApplication, in prj : EffectiveProject)
                : ResolvedProcess {
        init { var own : LockedImage := locked(self.image); }
        name := self.name;
        image := own.repository + '@' + own.digest;
        if self.lifecycle = Lifecycle::application
                then switchover := switchoverOf(app, self) endif;
        deadline := inSeconds(deadlineOf(self));
        secrets := self.secrets->collect(g | g.resolvedFor(self));
        placement := object ResolvedPlacement {
                eligibleNodes := contract.nodes->select(n | eligible(self, n))->collect(n
    | n.name);
        };
        dependencies := self.dependsOn->collect(e | edgeOf(e));
        environment := environmentOf(self, prj)->sortedBy(e | e.name);
}
```

Example 4: The Process Mapping, Abridged

3.4.2 The Edge Rule

A dependency edge names an `Application` and one of its `Surfaces`, for example platform-postgres and postgres. The query `edgeOf` finds the first `Process` in the cluster belonging to that `Application` that provides that `Surface`. The resolved edge records the `Process` address and the network peer the consumer may reach. If no `Process` provides the `Surface`, the `Platform Intent` external provider list is tried; this is how `auth` reaches a mail server outside the cluster. If neither has it, the run stops.

While a `Project` still runs at its old location, as the handover ledger records, its address and peer use that location's namespace and name, so a consumer can depend on it before it moves.

3.4.3 Environment Variables and Placeholders

A `Process`'s environment files consist of a base file and, optionally, one overlay per cluster. The resolution keeps base entries not replaced by the overlay for the `Platform Intent` cluster.

An entry may contain a placeholder, such as `DB_HOST=${dependency:platform-postgres.host}`. The query `Placeholder::entryFor` handles four kinds. A secret placeholder becomes a reference to a key of a secret the `Process` holds, so the value never appears in a file. A dependency placeholder takes the host or port of the `Process` edge from `edgeOf`. An exposure placeholder takes the URL, host or scheme of an `Exposure` anywhere in the cluster. An identity placeholder takes the `Process` name, namespace or Vault role. Text after a placeholder is kept, as in `AUTH_LOGIN_URL=${exposure:auth.public#url}/login`.

The runtime profile then adds its own variables. An author who writes one of them in an env file stops the run, because one name would have two values.

3.4.4 Network Peers

The ingress of a `Process` is a union of six sources (Table 6). Most look at other `Projects`: consumers whose edges name it, migrations of `Applications` whose database it holds, proxies and the metrics collector that the platform names. Each source yields tuples of rule, namespace, `Process` and port. Two objects are never equal in OCL, so a union of `PolicyPeer` objects would incorrectly keep a peer admitted twice; the union therefore runs on plain tuples, through `asOrderedSet()`, before the tuples become `PolicyPeer` objects.

3.4.5 The Path Plan

The path plan assigns each generated file its path as a `Deliverable` subclass that refers to the elements it generates from. The `Project` gets a namespace file, index, default-deny network policy, and the secret operator connection where an `Application` syncs secrets.

Each `Application` gets a directory with one file per resource kind: a workload file, a service account, RBAC, volume claims, a configuration map carrying `Release Gate` inputs as JSON, backups, a migration, a monitor, synced secrets, and a network-policy file with one policy per `Process`. Blue/green `Processes` add a `Canary` file, others a `Service` file; multi-replica `Processes` add a disruption budget. Empty files are not planned.

Each index lists the files of its directory that Flux applies. Network policies are generated but not yet applied, so no index lists them. Each `Exposure` gets a route file under its entry point. Each identity holding a secret gets a Vault policy and role.

Because paths are in the model, two files claiming one path are caught before any text is written; no path is chosen inside a Task 3 template.

3.5 The Trace and End Sections

A `Release Gate` member, route, scrape and every per-`Process` file refer to a `ResolvedProcess`, found with `p.resolveone(ResolvedProcess)` on the source `Process`, which the `Application` mapping has already mapped (Section 3.2).

Four values can only be computed once the rest of the model exists, so `end` sections, which run after the mapping body finishes, set them. The `Application`'s `Revision`, a digest of its complete element, names the release and appears in every generated `Canary`, and the migration file tag takes its first digits. The provenance render hash covers every input digest, and the order between `ReconcileUnits` refers to units created by the same mapping.

3.6 The Black-Box Library

A digest is a SHA-256 hash over canonical text. OCL lacks a byte-hashing operation, so the Java class `Hashing` provides three contextual operations. `digestOf` walks any model element reflectively, writes canonical JSON and hashes it. `textDigestOf` hashes a file's text as the production implementation does. `rawDigestOf` hashes a string as-is.

The `Revision` uses the third. Generated files carry it, so it must equal the production implementation's `Revision`, which is the digest of the `Application` element in that implementation's JSON form, a shape the

target metamodel does not have. The query `elementJson` therefore writes that element as text in OCL, from source and target model together: keys sorted, absent values omitted, every value spelled as the production implementation spells it. The black box only hashes the result, so every decision about what the `Revision` covers stays inside the transformation.

An operation taking a plain `EObject` is not found on a model element, because a model type does not conform to `EObject` in the compiler, so each call casts first: `p.oclAsType(ECORE::EObject).digestOf()`. The unit also cannot be named after the Java package, because `resolve` is a QVTo keyword; it registers as `dev.jorisjonkers.deploykit.emf.hashing`, by the runner outside Eclipse and by `plugin.xml` inside.

3.7 The Estate Pass

One generated job writes every `Project`'s Vault documents into the Secret Store. Its name is a digest over all of them, so changing any document starts a new job.

The estate pass, `estate(inout rendered : RD)`, runs once over all `Resolved Deployments` produced together. It collects every `VaultPolicyFile` and `VaultRoleFile`, writes each as canonical JSON in OCL, sorts them by file name, hashes the whole with `rawDigestOf`, and adds the job's four files and their index to the first model. It also adds the route for the delivery tool's events to the platform's reporter, which likewise belongs to no `Project`. The pass stops if two models carry different jobs, because one run has one `Platform Intent`.

3.8 Stops and Running the Transformations

Fifteen fatal assertions in the resolution stop the run when a value cannot be derived. Thirteen are in stop queries that a rule calls instead of returning an invalid value. For example, `locked` returns the lock entry for an image or calls `unlocked`, whose assert fatal ends the run with `E_UNLOCKED_IMAGE`. Two check `Release Gate` preconditions at the start of the gate mapping. The estate pass has one more, which stops it when two `Projects` carry different policy jobs.

An input can be missing: an image absent from the lock, an `Asset` whose file is missing or contains a placeholder, or a quantity in a unit the model does not read. A reference can name nothing: an edge no `Process` or provider serves, a placeholder naming nothing or a secret the `Process` does not hold, or a migration reaching no database. Some declarations cannot be honoured: a runtime-profile variable written by an author, a backed-up `Volume` without schedule or database engine, a blue/green `Application` without readiness, `Release Gate` or delivery policy, or a `Grant` without a secret store.

Where the specification defines an error code, the message contains it. An audience that no entry point carries has no stop because a constraint `E_NO_TIER_FOR_AUDIENCE` already refuses it before the run.

Three Java classes (`Lowering`, `Resolution` and `Estate`) run the transformations through QVTo's own `TransformationExecutor`. They register the black box, set configuration properties, pass the extents, and return the result or fail with the executor's diagnostic.

The build executes the transformations with QVTo's standalone executor on every code change; Eclipse launch configurations can also run them (Appendix B). The pipeline builds the extents from parsed and checked documents and writes every example's models under `target/parity/<example>/`. Appendix B.1 gives the steps.

4 Tests and Evaluation

The transformations are tested by running them on eight examples and comparing their output with oracle files. The oracles come from the production implementation processing the same input documents, so matching them shows both implementations derive the same values.

The build runs six test types on every change.

- **Lowered models.** For the four examples with shared declarations, the lowered model is written as JSON and compared byte-for-byte with the expected Effective Intent.
- **Dependency edges.** For each example, the resolved edges are exported as JSON and compared byte-for-byte with the expected file. These tests read only files the pipeline wrote, so they check the pipeline as a user would run it.
- **Revisions.** Each expected resolved file records its `Revision`, a digest over the `Application` element as

the production implementation writes it. Matching `Revisions` confirm that the same values were derived for that element (Section 4.1.1). The digest covers the element's own decisions, not the whole resolved model: the path plan, the network policies and the Vault documents fall outside it, which is what the complete-model and generated-file comparisons below are for.

- **Complete expected models.** `minimal` and `secrets` have full expected `Resolved Deployment` models, compared using EMF's structural equality (`EcoreUtil.equals`), which checks every attribute and reference.
- **Generated files.** Since Task 3 templates decide nothing, every wrong value they write from a `Resolved Deployment` becomes a wrong byte in a generated file. Tests generate the files from the transformation output and compare each with the committed output.
- **Interface tests.** JUnit tests exercise the resolution with the smallest project that resolves, a project not in the input, and an `Asset` whose file is missing, which must stop the run with the file name. The estate pass is tested with one job planned over two projects, a job name that moves when a document changes, no job without documents, and two projects that carry different policy jobs. The lowering is tested with eight cases of merging shared declarations.

The Java code around the transformations must reach full line and branch coverage and a 100QVTo code is verified only by the comparisons above.

4.1 Results

Table 7 lists the eight examples. All resolve successfully and every comparison passes, including the complete-model comparison of `minimal` and `secrets`. The full build runs 323 tests including 81 parity checks and passes.

	Counted in the resolved model							Equal to the expected output			
Example	**Apps**	**Proc.**	**Edges**	**Secr.**	**Vol.**	**Env.**	**Files**	**Edges**	**Revisions**	**Files**	**Model**
`minimal`	1	1	0	0	0	6	12	yes	1/1	yes	yes
`auth`	1	2	4	3	0	21	19	yes	1/1	yes	–
`data`	3	3	0	1	3	3	37	yes	3/3	yes	–
`delivery`	4	4	1	0	0	0	27	yes	4/4	yes	–
`edge`	2	2	0	0	0	0	13	yes	–	–	–
`knowledge-platform`	4	7	5	4	6	84	49	yes	1/1	yes	–
`observability`	2	2	0	0	0	0	15	yes	–	–	–
`secrets`	1	1	0	0	0	0	8	yes	–	–	yes

Table 7: The Resolved Examples and Their Comparisons (–: no committed oracle of that kind)

The `secrets` example holds no expected resolved file and no committed tree, but its complete expected model checks every attribute and reference of its whole resolution, where a `Revision` would check one element and a tree comparison the rendered files.

Between them, the examples exercise every row of Table 6. `minimal` alone exercises the common `Process` and `Exposure` rules (Table 5). The migration, placeholder and external-provider rules run in `auth`, which holds three secrets and reaches a mail server outside the cluster. The volume, backup and configuration-file rules run in `data`, which holds one volume of each durability class, a `Sidecar` and an `Asset`. The rolling and stop-start switchover rules run in `delivery`, which also has API access. The `secrets` example models the Secret Store itself, whose one `Process` is interrupted and runs under a stop-start switchover. The `knowledge-platform` example is the largest. The `edge` and `observability` examples provide the proxies and collectors the other examples look up. Provenance, the `Revision`, the generated files and egress run in every resolution.

4.1.1 Revision Evidence

An `Application`'s `Revision` is the digest of its whole element as the production implementation writes it: its `Processes` with image, user, deadline, probes, placement, secrets, volumes, backups, environment and edges; its `Exposures` and routes; its `Release Gate` and migration.

The transformation builds that text from its own `Resolved Deployment` (Section 3.6), so one different value,

one extra environment variable or one different order changes the digest. Ten `Applications` across five examples have a `Revision` in an expected file, and all ten match.

For these `Applications`, the `Revision` check verifies nearly every value of each element at once, where a hand-written model would have to check the fields one by one. A further test confirms the `Revision` moves when a decision changes: raising the memory in `minimal` from 256Mi to 512Mi changes it, but changing the snapshot capture time, which decides nothing, does not.

4.1.2 Generated-File Evidence

The generated files carry the values no `Revision` covers. The transformation output reproduces 167 committed files byte-for-byte, across five examples and the cluster-wide tree, including the estate pass job.

4.1.3 Thin Evidence

Two examples, `edge` and `observability`, have no `Revision`, no committed tree and no expected model. Their edge files contain no edges, so the evidence that they resolve correctly is that the run succeeds and that other examples, which look up their proxies and collectors, reproduce their expected files.

The `secrets` model is a reviewed output rather than an independent oracle. It flags any later change in that output, and the review of it found one gap in the specification: the Secret Store network policy admits nobody inbound even though every holder of a Secret may send to it.

The hand-written model of `minimal` from Task 1 was updated after hand-in to follow the metamodel changes of Appendix A.2. Its `Revision` was taken from the production implementation, so one field no longer checks independently.

4.1.4 Defects Found

Comparisons found a defect no single example had shown: the path plan placed cluster-wide route files under `apps/edge/`, which is also the directory of the project named `edge`. The `edge` example's files would have mixed with routes from every other `Project`. The fix moved all cluster-wide files under `estate/` in both implementations, and the committed trees now hold each file at its correct path.

4.2 Limitations

The oracles come from another implementation, so agreement shows two implementations of the specification decide the same, not that the specification itself is correct. Only the hand-written model of `minimal` was written directly from the specification.

Eight examples form a sample, on which every rule runs and agrees. The `knowledge` example, which models a `Project` being retired, is not one of them; its successor `knowledge-platform` is.

Checks across documents that remain in the transformation are stops, and a stop reports one problem without its location in the documents. Most such checks moved into constraints. The cross-document constraints still use `allInstances()`; moving them onto one root object is decided but not implemented.

QVTo caches mapping results but not queries, so `edgeOf` or a `Surface` address is recomputed wherever used. Several queries scan every `Process` in the cluster, so work grows with the square of `Process` count; for much larger clusters, hot queries would become mappings or precomputed properties, and capacity needs to be measured first.

A `Process` is eligible for a machine that can hold it alone; the resolution does not sum what all `Processes` request of one machine.

When two `Processes` provide the same `Surface`, the first in name order wins. The order between `ReconcileUnits` is set per `Project`, not per `Application`, because Flux orders whole units.

Each implementation hashes its own model form of the inputs, so their provenance digests differ. Every digest that reaches a generated file (the `Revision`, an `Asset` name and the policy job name) is reproduced.

The Eclipse run configuration is not tested: the build uses the standalone executor, and the configuration and `plugin.xml` registration are only run manually.

4.3 Reflection

The `Revision` took the most time. The OCL that writes the production implementation's JSON form of an `Application`, one field at a time, fills about one sixth of `resolution.qvto`, and every mistake in it showed only as a different hash.

Most rules are queries over the whole cluster, which OCL states directly, and the explicit trace made every reference between created elements visible, so QVTo suited the work. In return, QVTo recomputes those queries wherever they are used (Section 4.2).

The keywords `resolve`, `from` and `access` each cost time, as did two rules of the language. `resolve` prevented naming the black-box unit after its Java package, and `from` and `access` forced two metamodel features to be written `_from` and `_access`. A multi-statement `if` needs braces. Inside an object expression, `result` refers to the enclosing mapping result, not the object being created.

With hindsight, the evidence should have been planned with the examples from the start. The expected model of `secrets` and the parity checks for `knowledge-platform` were only added at the end, when the examples without a comparison were identified. The Task 1 model of `minimal` should have been kept unchanged beside the updated one, so the one independent oracle would remain independent.

A Changes to the Task 1 Metamodels

This appendix lists the changes between the Task 1 metamodels and those the Task 2 transformations read and write. The comparison checked each class and feature in the `.ecore` files.

A third metamodel for the `Pinned input set` was added (Appendix A.1). The target metamodel gained derived values and twelve kinds of generated file (Appendix A.2), and the source metamodel gained declarations the other examples need (Appendix A.3). Four grammars were added (Appendix A.4).

No class was removed. The Task 1 hand-in showed OCL helper definitions such as `levelsAbove` and `derivesDatabase` as derived attributes in the diagram. They remain OCL definitions in `project-intent.ocl` and were never Ecore features.

A.1 The Pinned Inputs Metamodel

The `pinnedinputs` package holds the documents the resolution reads beside the source models (Section 1.3).

The node contract is generated from the platform team's machine descriptions. Other programs publish the images lock and the snapshot. A `Project` build writes its migration proof.

The committed descriptor that compares the source metamodel's structure with the implementation schemas keeps the two aligned. With the pinned input classes in the source metamodel, that comparison would find documents the schemas do not hold. No target metamodel class refers to them: the `Resolved Deployment` records the values read from them, and their digests.

Figure 3 shows the metamodel. Table 8 shows which rules read each class.

Class	Read by
`NodeContract`, `Node`, `Allocatable`, `Gpu`, `Disk`	`eligible`: memory and CPU as quantities such as 53500m, and site, architecture, capabilities, GPUs, disks.
`ImagesLock`, `LockedImage`	`locked`: every image of a `Process`, `Sidecar`, backup method, migration runner and Vault policy job.
`ClusterState`, `Binding`	`boundOf`: the machine that already holds a `Volume`.
`CurrentPlacement`, `RollbackBackup`	Not read by the resolution. The snapshot carries them for the delivery process that Task 3 serves.
`MigrationProof`, `ProvenApplication`	`migrationOf` and `gate`: the schema version tested against, and whether it holds a non-transactional change.
`AssetFile`	`Asset::resolvedFor`: a configuration file's text. The pipeline creates one per file a `Project` names. It is not parsed.

Table 8: The Classes of the Pinned Inputs Metamodel and the Rules That Read Them

Two attributes are plain strings, an exception to Task 1's rule of enumerations over free-form strings. A `Node`'s disk medium is not the source metamodel's `Media` enumeration because the node contract lists media no `Process` may request, such as sdcard. The resolution compares the spelling instead, and does the same for architecture. The values come from generated documents, not from an author, so the risk of an author's typo does not apply.

A.2 Changes to the Target Metamodel

Both halves of the target metamodel grew: the decisions the resolution derives for each `Application`, and the generated-file entries of the path plan. The policies those decisions read arrive as inputs from the `Platform Intent`, and the metamodel records only the derived output.

The root class is unchanged, but there is now one instance per `Project` instead of one for the whole cluster (Section 1.3).

Decisions. Figure 4 shows every decision class that is new or changed. Table 9 gives the reason for each.

Task 1 named many decisions in prose but gave classes only to what its worked example needed. The other examples also need migrations, backups that connect to their database, `Secrets` that reach the cluster, API access, `Sidecars`, configuration files, and two cluster-wide values: the Vault policy job and the route of the delivery tool's events.

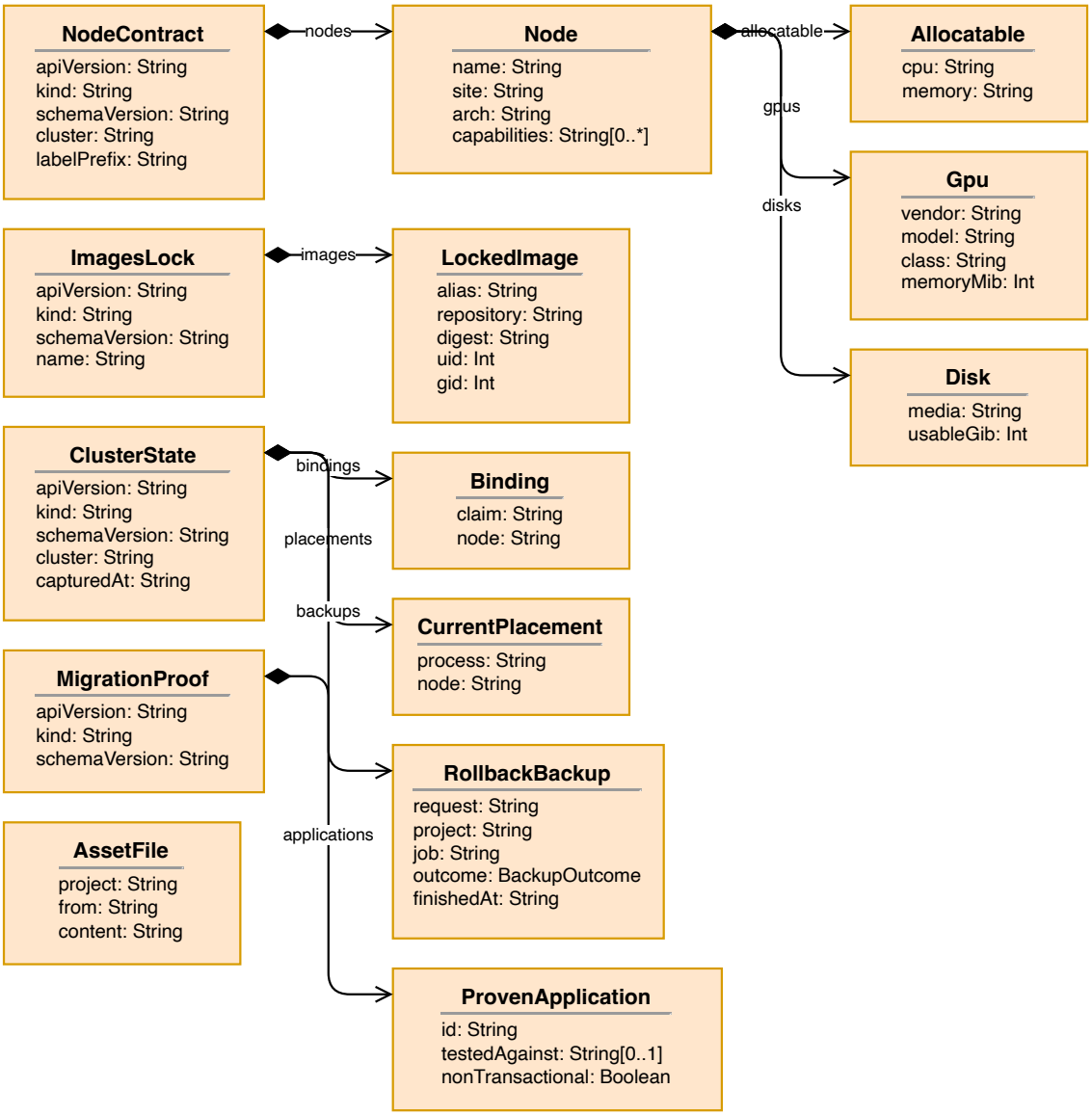


Figure 3: The Pinned Inputs Metamodel. Every class is new in Task 2

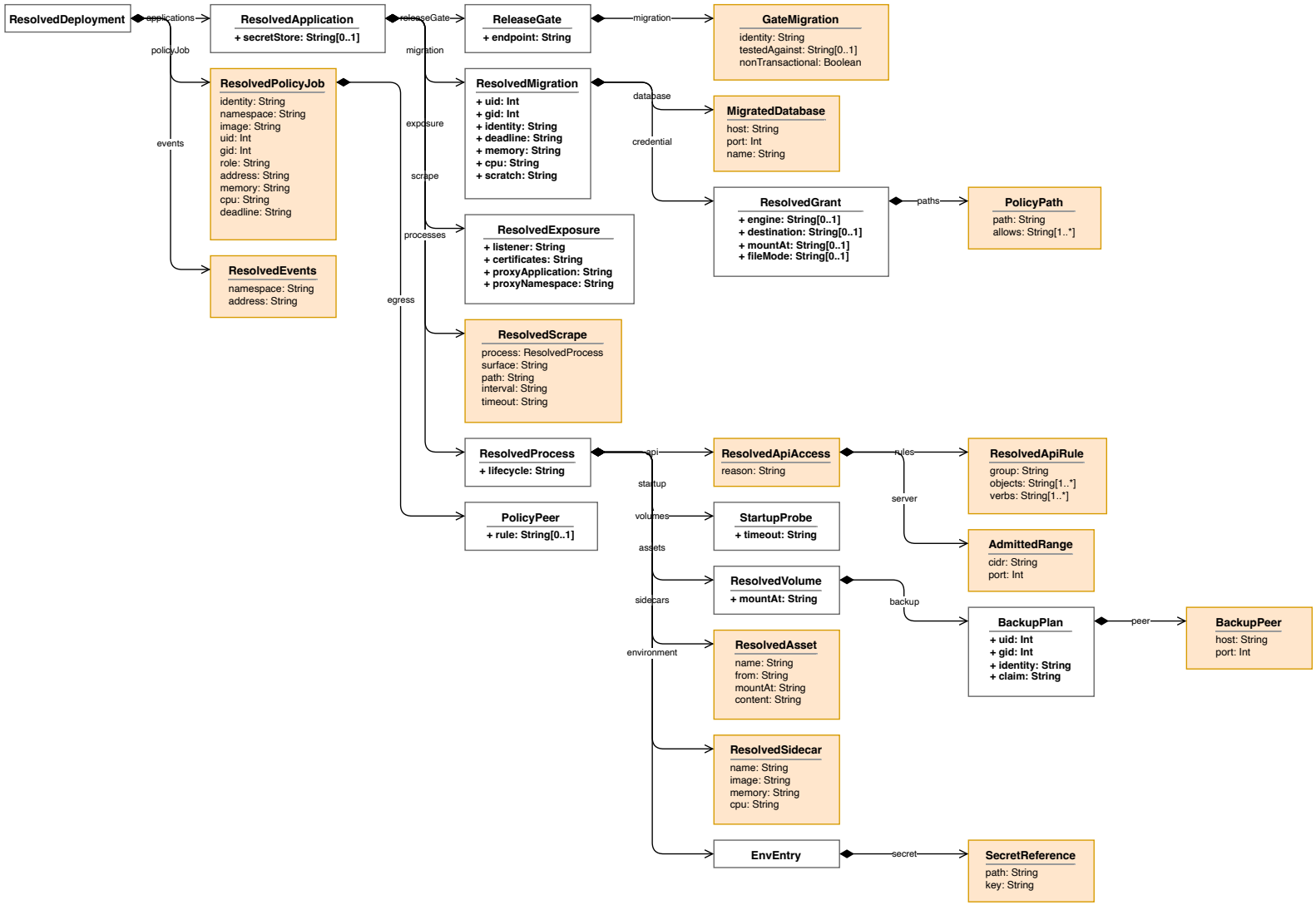


Figure 4: Decision Classes of the Target Metamodel That Changed. Orange marks a new class; white a Task 1 class, showing only new features in bold. Arrows are containment; a class contained in several places is drawn under its first container only

Class	Change	Reason
`ResolvedDeployment`	added `policyJob`, `events`	The Vault policy job and the route for delivery tool events belong to the whole cluster; every `Project` model carries the same job; the estate pass plans their files.
`ResolvedPolicyJob`, `ResolvedEvents`	new	The job's identity, image, Vault role, resources and egress; the events receiver's namespace and address.
`ResolvedApplication`	added `secretStore`, `scrape`	The secret store address where the operator syncs from, and how the metrics stack scrapes the `Application`, which the revision element names.
`ResolvedScrape`	new	The scraped `Process`, `Surface`, path, interval and timeout.
`ReleaseGate`	added `endpoint`, `migration`	An address that every `Canary` webhook calls; the migration the gate starts and may undo.
`GateAnalysis`	added `errorRate`, `latency`	The thresholds of the two analysis checks, from the `Platform Intent`.
`GateMigration`	new	The migration's identity and what its proof records.
`ResolvedMigration`	added `uid`, `gid`, `identity`, `deadline`, `memory`, `cpu`, `scratch`, `database`, `credential`, `egress`	Task 1 named the identity, owner role and deadline in prose. A migration job also needs the database address, the owner credential that only it holds, and a network policy.
`MigratedDatabase`	new	The database host, port and name.
`ResolvedProcess`	added `lifecycle`, `api`, `assets`, `sidecars`	Whether the `Process` serves, runs once or prepares (which the revision element names); its API access, configuration files and `Sidecar`s.
`ResolvedApiAccess`, `ResolvedApiRule`, `AdmittedRange`	new	The API rules a `Process` holds; the address range a policy admits: the API server for API access, the backup off-cluster destination.
`ResolvedAsset`, `ResolvedSidecar`	new	A configuration file with a content-derived name; a `Sidecar` with a locked image and resources.
`ResolvedGrant`	`path` and `access` optional; added `engine`, `paths`, `destination`, `mountAt`, `fileMode`	A transit or database `Secret` has no single path; it is recorded as the Vault paths its policy covers. A synced `Secret` needs a Kubernetes `Secret` copied to its mount.
`PolicyPath`	new	One path of a Secret policy and the operations allowed on it.
`ResolvedVolume`	added `mountAt`	The workload template needs the mount path.
`BackupPlan`	added `user`, `group`, `identity`, `claim`, `egress`, `peer`, `credential`, `destinations`	A backup runs as its own identity with its own claim and network policy; it connects to the `Process` it dumps and copies off-cluster with a credential.
`BackupPeer`	new	Where a backup connects, and the credential it logs in with.
`EnvEntry`	`value` optional; added `secret`	A secret placeholder becomes a reference, so the value never appears in a file.
`SecretReference`	new	The secret path and key a variable reads.
`StartupProbe`	added `timeout`	The specification gives the startup probe the `Platform Intent` probe timeout.

Class	Change	Reason
`ResolvedExposure`	added `listener`, `certificates`, `proxyApplication`, `proxyNamespace`	The entry point's protocol and certificate source; the proxy whose middleware the routes name.
`PolicyPeer`	added `rule`; `process` optional	Which rule admitted the peer, so two rules admitting one peer stay two entries, as in production; a peer such as cluster DNS has a namespace but no `Process`.

Table 9: Changes to the Decision Classes of the Target Metamodel

Generated Files. Figure 5 shows every subclass of `Deliverable`, up from Task 1's nine to 21.

Workload, Canary and identity files, which Task 1 tied to one `Process`, now list all of an `Application`'s `Processes`, because the specification puts one file per `Application` holding all its `Process` resources. For the same reason, the network policy file holds one `ProcessPolicy` per `Process`, and the Secret file refers to its `Application` and holds one `SecretHolder` per identity, since a Backup identity holds Secrets too. Task 1's worked example had one `Process`, so this difference did not show.

`MonitorFile` gained timeout and kind: a blue/green `Process` is scraped through its Pods; any other through its `Service`.

The twelve new file kinds are Services, volume claims, configuration maps, backups, disruption budgets, migrations, RBAC, the secret operator connection, Vault policies, Vault roles, the policy job and the events route.

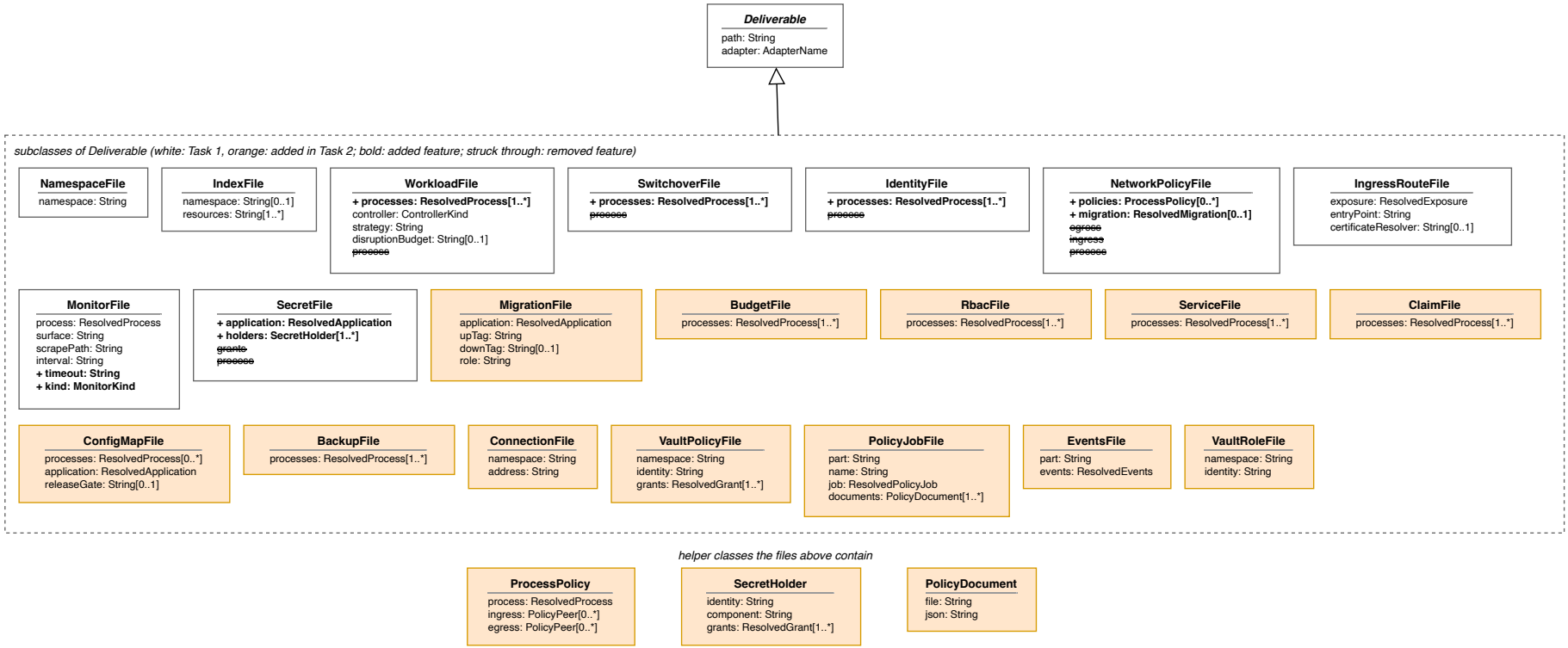


Figure 5: The Generated-File Classes of the Target Metamodel, Task 1 Against Task 2

The hand-written model of `minimal` from Task 1 was updated with these changes. Its files refer to the `Process` through `processes`, it gained the configuration map that carries the `ReleaseGate` inputs, and its `Revision` was set to the production implementation's (Section 4.1.3).

A.3 Changes to the Source Metamodel

The source metamodel kept every class. Table 10 lists what was added or changed.

Most additions are declarations in the `Platform Intent` that a derivation needs and Task 1 left implicit. A rule needing a value the `Platform Intent` does not declare would otherwise contain it as a constant.

Class	Change	Reason
`Process`	added `api` (`ApiAccess`, `ApiRule`, `ApiVerb`)	The `Process` declares the Kubernetes API access it needs, with a reason; the platform's delivery tools need it.
`Platform`	added `apiAccess` (`ApiAccessPolicy`)	Which `Application`s may hold API access, and the API server address range.
`Platform`	added `secretStore`, `telemetry` (`TelemetryPolicy`), `policyJob` (`VaultPolicyJob`)	References to the secret store, the telemetry collector and the metrics stack, whose addresses the resolution finds in their `Project`s; Vault policy job image, role and resources.
`DeliveryPolicy`	added `gate`, `reporter`	References to the `Release Gate` and the events receiver `Application`s.
`AnalysisPolicy`	added `errorRate`, `latency`	The thresholds the `Release Gate` checks.
`Substrate`	added `clusterDns`	The namespace every egress policy admits for DNS.
`EnginePolicy`	added `surface`	The port a database engine backup connects to.
`OffClusterCopy`	added `egress` (`DestinationRange`)	The address ranges a backup copy goes to.
`HandoverLedger`	added `serving` (`ServingLocation`)	Where a `Project` not yet moved still runs, so an edge to it resolves.
`RenderedArtifacts`	added `pullSecret`	Used by delivery, not by the resolution.
`Placeholder`	added `suffix`	Text after a placeholder, as in `${exposure:auth.public#url}/login`.
`Grant`	`rotation` required	Every `Grant` states how it tolerates rotation, which decides its restart targets.
`Volume`	`size` required	A claim cannot be generated without a size.
`EffectiveProject`	removed `apiVersion`, `kind`, `schemaVersion`	The lowered model is written by the lowering, not by an author, so it has no document header.

Table 10: Changes to the Source Metamodel

A.4 Grammars for the Pinned Inputs

The pinned inputs use grammars rather than a Java YAML reader or hand-written XMI, so both implementations read the same YAML files and their results can be compared.

Four grammars were added beside the Project and Platform grammars: NodeContract, ImagesLock, ClusterState and MigrationProof. Each inherits the Project grammar, reusing its terminals, indentation tokens and scalar rule, and adds only the rules for its own document. As with the `Platform Intent`, the file name selects the grammar: `node-contract.yml`, `images.lock.yml`, `cluster-state.yml` and `migration-proof.yml`.

One input file changed: the Project grammar scalar terminal cannot contain a colon, so the snapshot's capture time is now quoted, as digests and URLs already were. The production implementation reads the same value either way.

Example 5 shows the smallest grammar.

```
grammar dev.jorisjonkers.deploykit.emf.syntax.ClusterState
        with dev.jorisjonkers.deploykit.emf.syntax.ProjectIntent
import "https://jorisjonkers.dev/deploy-kit/pinned-inputs/1"

ClusterState returns ClusterState:
        (('apiVersion' ':' apiVersion=Text)
        & ('kind' ':' kind=Text)
        & ('schemaVersion' ':' schemaVersion=Text)
        & ('cluster' ':' cluster=Text)
        & ('capturedAt' ':' capturedAt=Text)
        & ('bindings' ':' BEGIN (DASH BEGIN bindings+=Binding END)* END)
        & ('placements' ':' BEGIN (DASH BEGIN placements+=CurrentPlacement END)* END)
        & ('backups' ':' BEGIN (DASH BEGIN backups+=RollbackBackup END)* END)?);

Binding returns Binding:
        (('claim' ':' claim=Text)
        & ('node' ':' node=Text));
```

Example 5: The Grammar of the Cluster-State Snapshot, Abridged

B The Hand-in Archive

The archive holds `report.pdf`, a `README.md` with the steps from Appendix B.1, and two folders.

`task-2-transformations/` is an Eclipse project for reading the models (Table 11), imported via `File > Import > Existing Projects into Workspace`. It contains no Java, so the transformations do not run from it: the resolution and estate pass call the `Hashing` library, which the implementation plug-in registers.

`implementation/` holds the EMF implementation and the examples the tests read.

Path	Contents
`model/`	The source, pinned-inputs and target metamodels.
`transforms/`	`lower.qvto`, `resolution.qvto` and `estate.qvto`.
`examples/platform/`	The `Platform Intent` and the pinned inputs every example reads.
`examples/<example>/authored/`	The `Project Intent` with its env and configuration files.
`examples/<example>/resolution.intent.xmi`	The intent extent: authored and lowered `Projects`, plus `Platform Intent`.
`examples/<example>/resolution.pinned.xmi`	The pinned extent.
`examples/<example>/<project>.resolveddeployment`	The `Resolved Deployment` the transformation wrote.
`examples/<example>/dependencies.json`	The edges exported from it, beside `expected-dependencies.json`, a copy of the oracle `expected/dependencies.json`.
`examples/minimal/minimal.hand-written.resolveddeployment`	The hand-written target model from Task 1, updated in Appendix A.2.
`examples/secrets/secrets.expected.resolveddeployment`	The complete expected model of `secrets` (Section 4.1.3).

Table 11: The Models Project of the Archive, One Folder per Example of Table 7

Every XMI declares its metamodel in `xsi:schemaLocation`, so Eclipse opens it in the Sample Reflective Ecore Model Editor without a registered package.

B.1 Running the Transformations

1. In `implementation/emf`, run `./mvnwcleanverify` with JDK 21. It builds the implementation, runs every test, and writes each example's models and edges under `bundles/cli/target/parity/<example>/`. It takes about four minutes.
2. In Eclipse Modeling Tools with the QVT-Operational SDK, import `implementation/emf` via `File > Import > Existing Maven Projects`.
3. Run `resolution.launch` in `dev.jorisjonkers.deploykit.emf.resolve`. It resolves `minimal` from the extents step 1 wrote.

References

[1] P. Merle and F. Petrillo. Visualizing Cloud-native Applications with KubeDiagrams. In *Proc. VISSOFT 2025*, pp. 106–116, 2025. Tool: https://github.com/philippemerle/KubeDiagrams.

[2] KubeOrch, visual Kubernetes workflow orchestrator. https://github.com/KubeOrch (repositories `core` and `ui`), visited 15 October 2026.

[3] kubert, a kubectx and kubens alternative. https://github.com/idebeijer/kubert, visited 15 October 2026.

[4] L. Burgueño, J. Cabot and S. Gérard. The Future of Model Transformation Languages: An Open Community Discussion. *Journal of Object Technology* 18(3), 7:1–11, 2019.

[5] M. van Amstel, S. Bosems, I. Kurtev and L. Ferreira Pires. Performance in Model Transformations: Experiments with ATL and QVT. In *Proc. ICMT 2011*, LNCS 6707, pp. 198–212, 2011.

[6] Object Management Group. *Meta Object Facility Query/View/Transformation Specification*, version 1.3, 2016.